

Đa hình

Ưu điểm

- Tính linh hoạt: Dễ dàng thay đổi và mở rộng code
- Khả năng tái sử dụng code: Viết code một lần, dùng nhiều kiểu đối tượng
- Giảm sự phụ thuộc: Code ít phụ thuộc vào kiểu dữ liệu cụ thể
- Dễ bảo trì: Thay đổi ở 1 nơi ít ảnh hưởng nơi khác
- Tính trừu tượng hoá: Tập trung vào "cái gì" thay vì "như thế nào"

Kế thừa hỗ trợ Đa hình

- Lớp con kế thừa phương thức từ lớp cha: Có thể ghi đè phương thức
- Tham chiếu kiểu lớp cha: Có thể trỏ đến đối tượng lớp con
- Gọi phương thức trên tham chiếu cha: Phương thức thực thi là của đối tượng con
- Tính đa dạng của đối tượng: Một tham chiếu có thể "hành xử" khác nhau tùy thuộc vào đối tượng nó trỏ tới

Khác biệt với Kế thừa

- Mục đích chính
 - Đa hình: Thực hiện các hành động khác nhau dựa trên kiểu đối tượng
 - Kế thừa: Tạo ra mối quan hệ "là một" (is-a), tái sử dụng thuộc tính và phương thức
- Cơ chế hoạt động
 - Đa hình: Dựa vào ghi đè phương thức và liên kết động
 - Kế thừa: Dựa vào việc lớp con nhận các thành viên từ lớp cha
- Mối quan hệ
 - Đa hình: Một đối tượng có thể có nhiều "hình dạng" (kiểu)
 - Kế thừa: Tạo ra cấu trúc phân cấp lớp
- Tính bắt buộc
 - Đa hình: Không bắt buộc phải có kế thừa (nhưng kế thừa thường được sử dụng để đạt được đa hình)
 - Kế thừa: Bắt buộc phải có mối quan hệ cha-con giữa các lớp